House Robber III (LeetCode 337)

> **House Robber III (LeetCode 337)** is one of the most important Tree DP problems because it teaches the general pattern of Dynamic Programming on Trees.

Most students know DP on arrays:

```
dp[i] = answer for index i
```

For trees, the idea changes to:

```
dp[node] = answer for the subtree rooted at node
```

This single change is the foundation of almost every Tree DP problem.

Part 1 : What is Tree DP?

Before learning House Robber III, let's understand what "DP on Trees" actually means.

DP on Arrays

Suppose

```
1 2 3 1
```

House Robber I
At every index, you ask

"What is the best answer considering houses from here onward?"

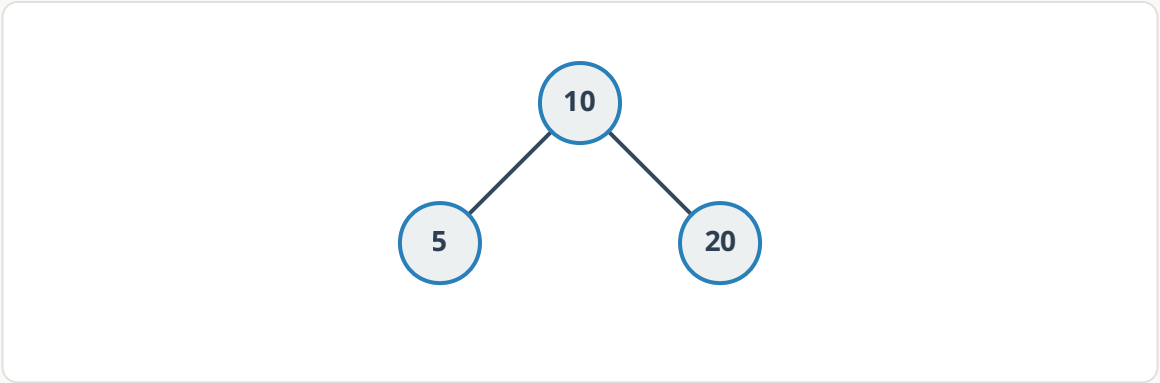
State
`dp[i]` means **Maximum money from index i to end.**

Notice

The answer for `i` depends on `i+1` and `i+2`.

DP on Trees

Now suppose



There is no `i+1`. There is no `i-1`.

Instead, every node has children. So the state changes.
Instead of `dp[i]` we think `dp[node]`.

Meaning

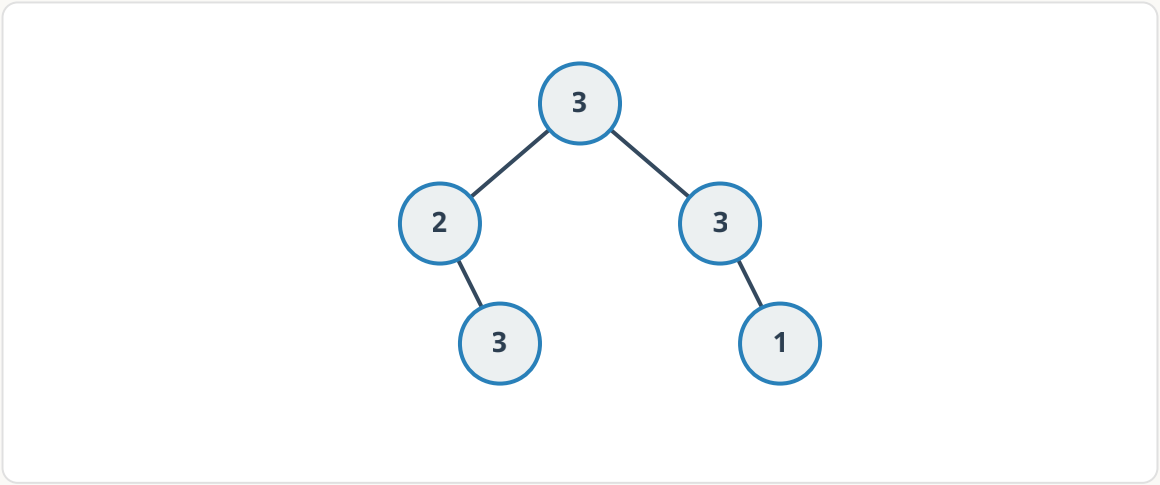
"Best answer for the subtree rooted at this node."
This is the biggest mental shift.

Step 1 : Read the Question Carefully

House Robber III

If you rob a node, you CANNOT rob its children.

Tree



If we rob `3 (root)`, then `2` and `3` cannot be robbed.

Immediately ask: **What decision am I making at every node?**

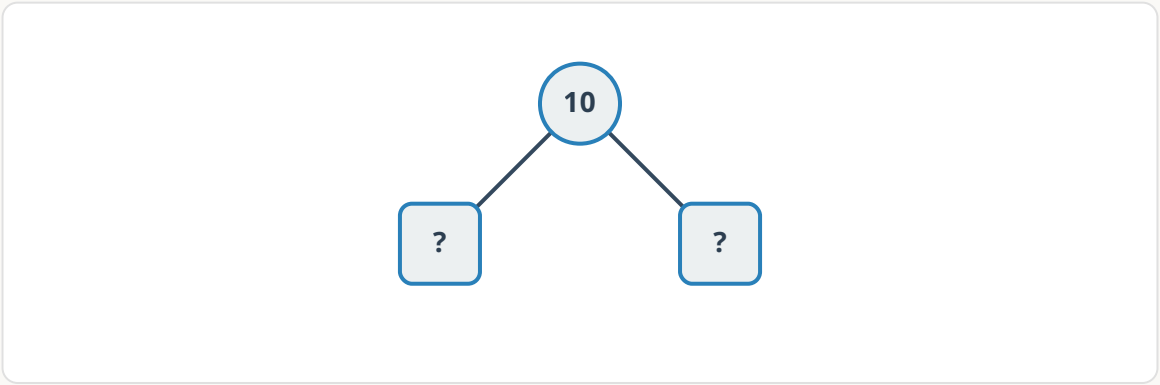
Answer: Only two choices.
Choice 1: Rob this node.
Choice 2: Don't rob this node.

> *Whenever you see **Take** OR **Don't Take**, think **DP**.*

Step 2 : Identify the State

Question: What information is needed from every subtree?

Suppose



Parent asks: *Should I rob myself?*

To answer that, the parent needs TWO answers from each child.

- If child is robbed: maximum money?
- If child is not robbed: maximum money?

Therefore, each subtree returns TWO values.
Instead of `dp[node]`, we use `dp[node][0]` and `dp[node][1]`, or `pair<int,int>`.

Meaning

- `0` -> don't rob this node
- `1` -> rob this node

This is the entire DP state.

Step 3 : Understand the Meaning

Suppose node `10` returns `{25 , 40}`.

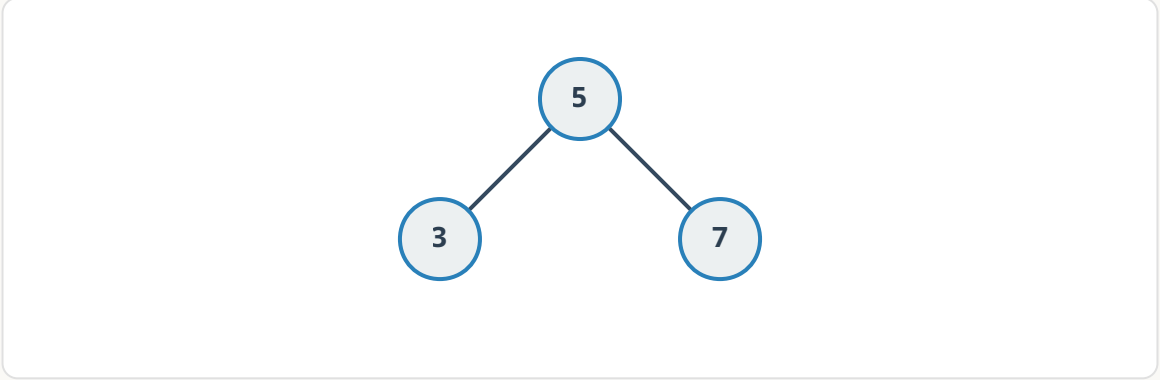
Means:
25 = Maximum money IF `10` is NOT robbed
And
40 = Maximum money IF `10` IS robbed

Notice: This answer already includes the whole subtree.

Step 4 : Think Bottom-Up

Tree DP almost always works: **Bottom ↓ Top**
Because Parent depends on children.

Example

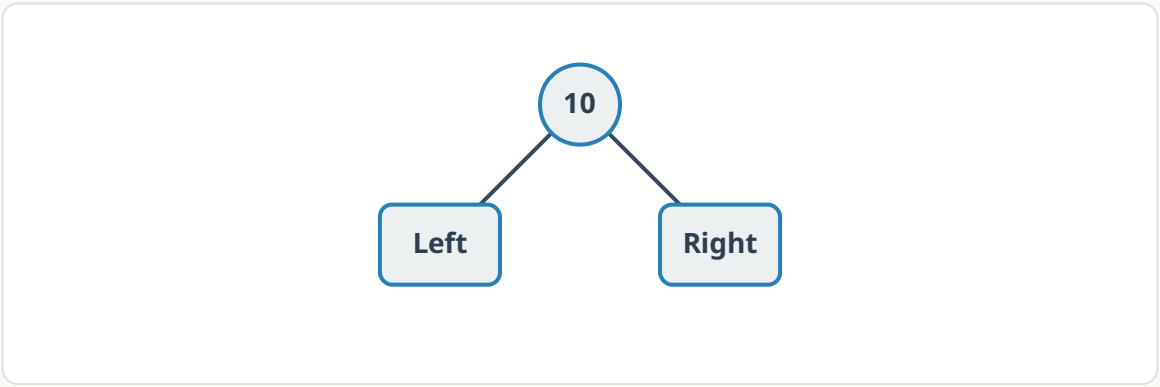


Can we compute answer for `5` first? **No.**
Need answers from `3` and `7`.

Therefore, use **Postorder DFS**: `Left` → `Right` → `Node`

Step 5 : Derive the Transition

Suppose Current node



Children already solved.
Left child returns `{20 , 30}`
Meaning: `don't rob left = 20`, `rob left = 30`

Right child returns `{15 , 40}`

Now compute current node.

Case 1

We **ROB** current node. `Rob 10`.
Can we rob children? No.
So Take: **Current value + Left NOT robbed + Right NOT robbed**

```
rob = node->val + left.notRob + right.notRob
```

Example: *10 + 20 + 15 = 45*

Case 2

Don't rob current node.
Now children become independent. Left can be robbed or not robbed. Take maximum.
Same for right.

```
notRob = max(left.rob, left.notRob) + max(right.rob, right.notRob)
```

Example: *max(20,30) + max(15,40) = 70*

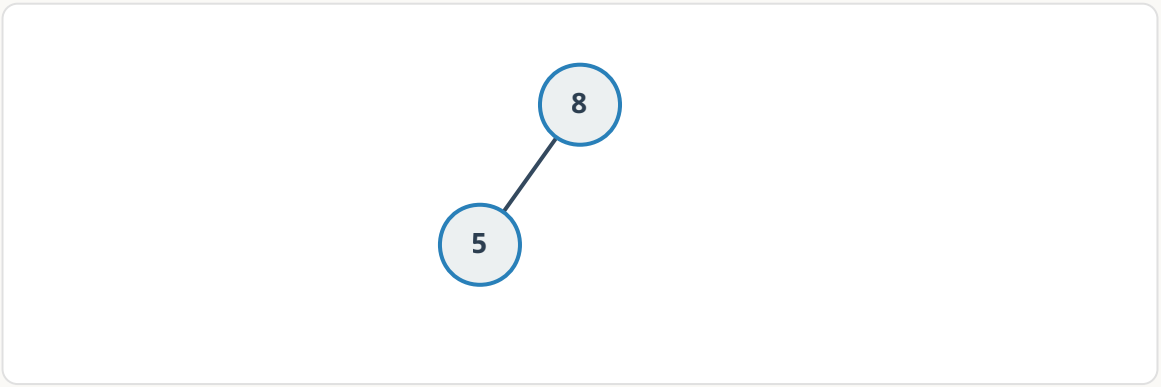
So current node returns `{70 ,45}`

> **THIS IS THE DP RELATION**
>
> ```
> rob = node->val + left.notRob + right.notRob
> notRob = max(left.rob, left.notRob) + max(right.rob, right.notRob)
> ```

Everything comes from the question itself.

Step 6 : Why is this DP?

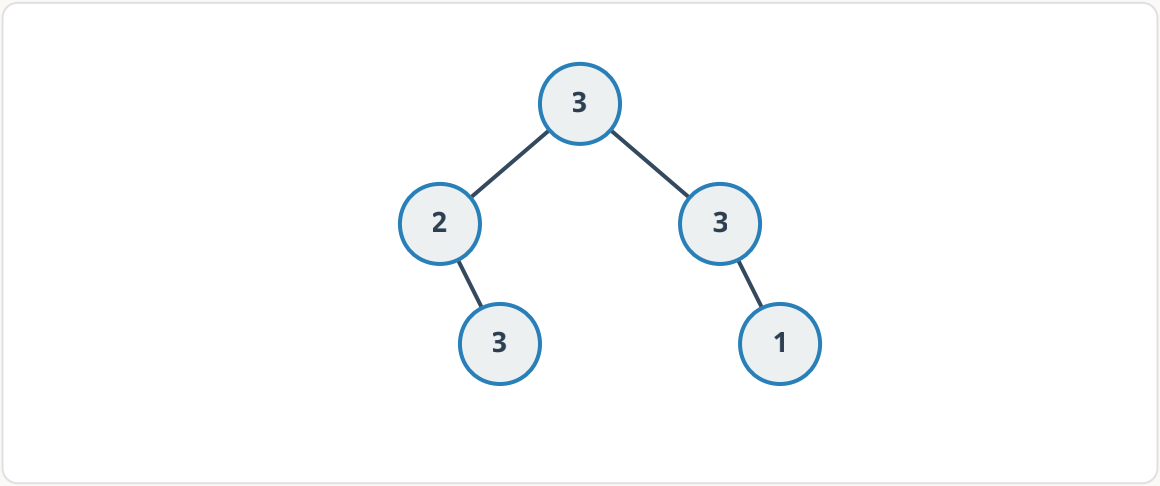
Suppose



Node `5` is solved once. Its answer `{notRob, rob}` is reused by parent.
Exactly like `dp[i]` in arrays.
Difference is only `dp[node]` instead of `dp[index]`.

Step 7 : Complete Dry Run

Tree



- **Leaf 3:** No children. If robbed: `3`. If not robbed: `0`. Returns `{0,3}`.
- **Leaf 1:** Returns `{0,1}`.
- **Node 2:** Left `nullptr` `{0,0}`. Right `{0,3}`.
  Rob: $2 + 0 + 0 = 2$
  Don't rob: $max(0,0) + max(0,3) = 3$
  Returns `{3,2}`.
- **Node 3:** Right child `{0,1}`.
  Rob: $3 + 0 + 0 = 3$
  Don't rob: `1`
  Returns `{1,3}`.
- **Root:** Left `{3,2}`. Right `{1,3}`.
  Rob: $3 + 3 + 1 = 7$
  Don't rob: $max(3,2) + max(1,3) = 6$
  Return $max(7,6) = 7$.

Correct.

Generic Tree DP Framework

Whenever you encounter a Tree DP problem, follow this checklist:

1. **What does each node represent?**
   Usually, the answer for the entire subtree rooted at that node.

2. **What decisions exist at each node?**
   For House Robber III, the decisions are:
   - Rob the current node.
   - Don't rob the current node.
3. **What information does the parent need from its children?**
   The parent must know both possibilities for each child:
   - Maximum value if the child is robbed.
   - Maximum value if the child is not robbed.
4. **What should the DFS return?**
   Return exactly the information the parent needs. Here, that's a pair: `{notRob, rob}`
5. **Can I compute the current node after computing the children?**
   If yes, use postorder traversal (Left → Right → Node).
6. **Write the transition from the children's answers.**

```
rob =
    node->val
    + left.notRob
    + right.notRob;
notRob =
    max(left.rob, left.notRob)
    + max(right.rob, right.notRob);
```

Final Pattern to Remember

Most Tree DP problems follow this structure:

```
State(node)
{
    // 1. Base case
    if(node == nullptr)
        return BaseState;

    // 2. Solve left subtree
    LeftState = State(node->left);

    // 3. Solve right subtree
    RightState = State(node->right);

    // 4. Combine children's states
    CurrentState = Transition(LeftState, RightState, node);
```

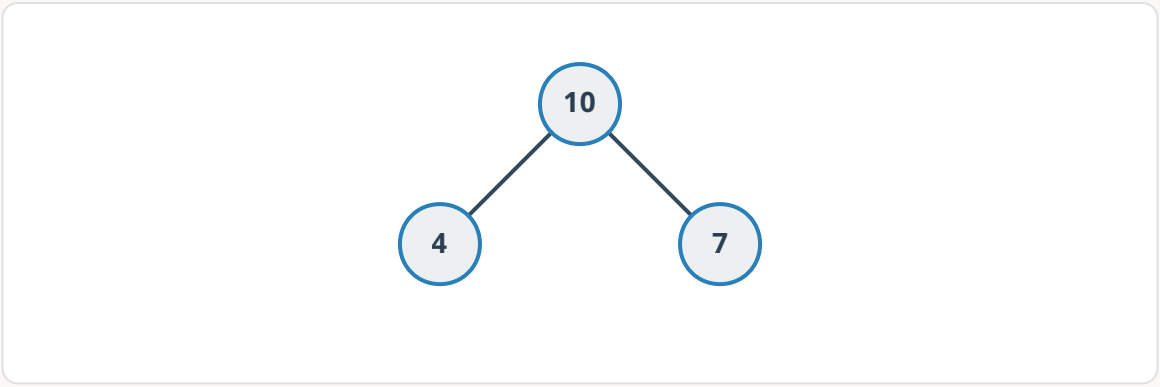
```
    // 5. Return the state of this subtree
    return CurrentState;
}
```

```
pair<int,int> dfs(TreeNode* root)
{
    // Empty subtree contributes nothing.
    if(root == nullptr)
        return {0,0};

    // Solve left subtree first.
    pair<int,int> left = dfs(root->left);

    // Solve right subtree.
    pair<int,int> right = dfs(root->right);

    //---------------------------------------------------------------
    // OPTION 1
    //
    // Rob current node.
    //
    // Children CANNOT be robbed.
    //---------------------------------------------------------------

    int robCurrent =
        root->val
        + left.first
        + right.first;

    //---------------------------------------------------------------
    // OPTION 2
    //
    // Don't rob current node.
    //
    // Every child independently chooses its better option.
    //---------------------------------------------------------------

    int notRobCurrent =
        max(left.first, left.second)
        + max(right.first, right.second);

    // Return DP state for this subtree.
    return {notRobCurrent, robCurrent};
}
```

```
int rob(TreeNode* root)
{
    pair<int,int> ans = dfs(root);

    // Root may or may not be robbed.
    return max(ans.first, ans.second);
}
};
```

Visual Meaning of the Pair

Suppose this subtree:



After solving it, DFS returns: `{25, 21}`

This does not mean: $25 + 21$

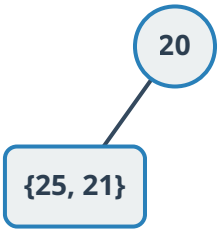
It means:

RETURNED VALUE	MEANING
25	Maximum money if 10 is NOT robbed
21	Maximum money if 10 IS robbed

The parent uses these two values to decide its own answer.

How the Parent Uses It

Suppose we have:



If the parent (20) decides to rob itself:

```
robCurrent = 20 + left.first;
```

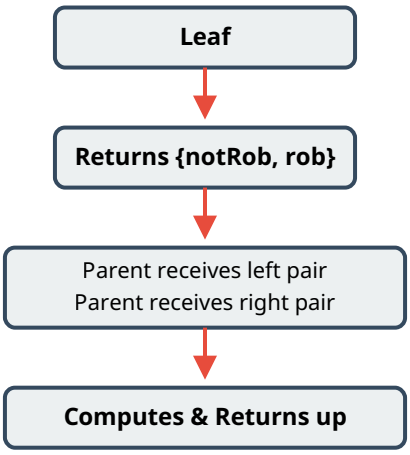
Why `left.first` ?
Because `left.first` means *"Child is NOT robbed."* which is required since the parent is robbed.

If the parent does not rob itself:

```
notRobCurrent = max(left.first, left.second);
```

Now the child is free to choose whichever option gives more money.

The Entire Tree DP Flow



Answer = `max(notRobRoot , robRoot)`

Tree DP Pattern to Remember

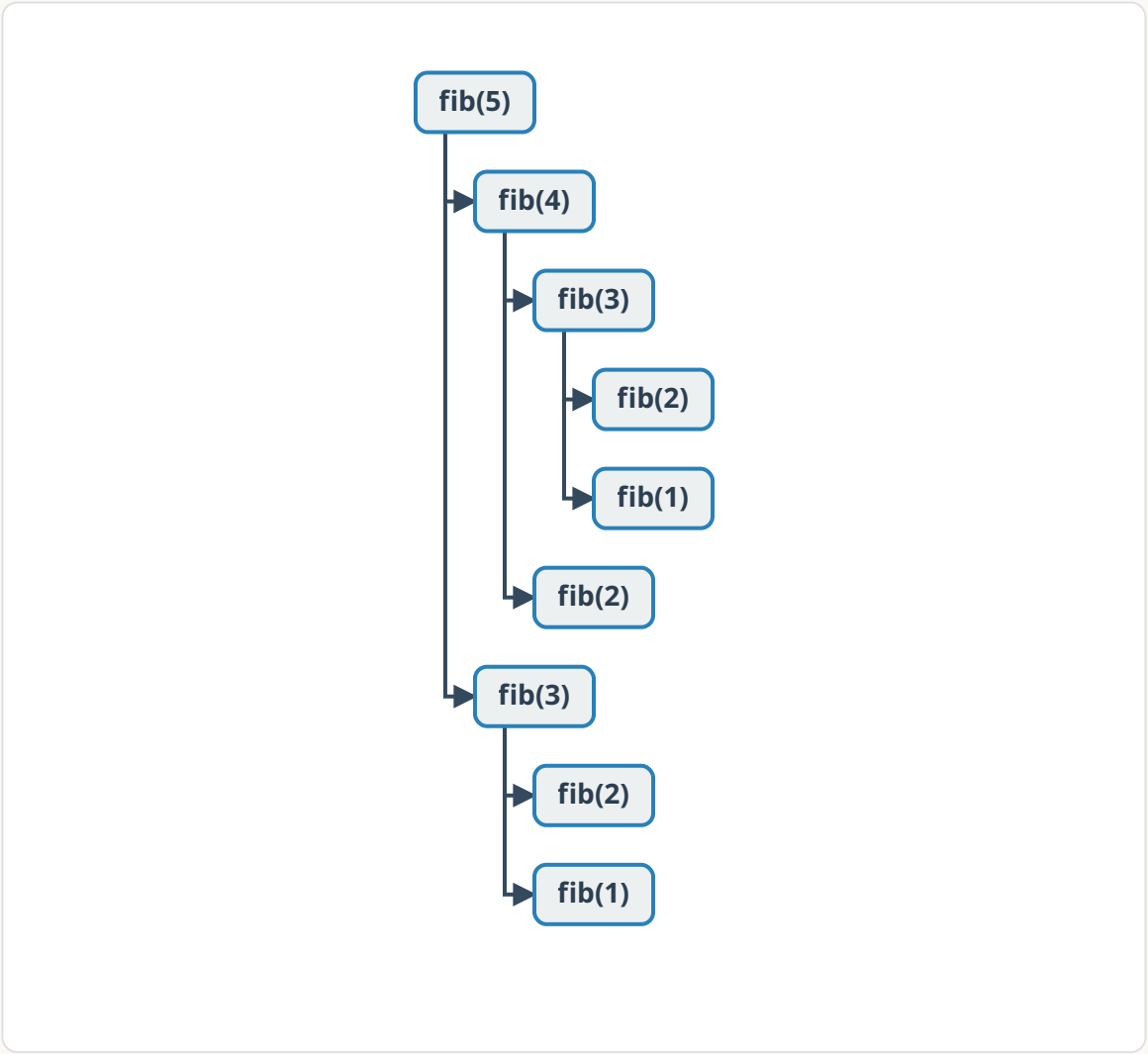
Every Tree DP problem can be approached with these 5 questions:

- **What is the DP state of one node?**
  Here: `{notRob, rob}`.
- **What information does the parent need from its children?**
  Both possibilities (robbed and not robbed).
- **Can the current node be computed from its children's answers?**
  Yes.
- **Which traversal computes children before the parent?**
  Postorder DFS (Left → Right → Node).
- **What should the DFS return?**
  The DP state for the current subtree.

Once you learn to answer these five questions, you'll find that most Tree DP problems—such as House Robber III, Binary Tree Cameras, Maximum Path Sum, and Distribute Coins in Binary Tree—follow the same bottom-up pattern with a different state definition and transition.

First, what is an overlapping subproblem?

In normal DP, the same state is computed multiple times.
Example: Fibonacci



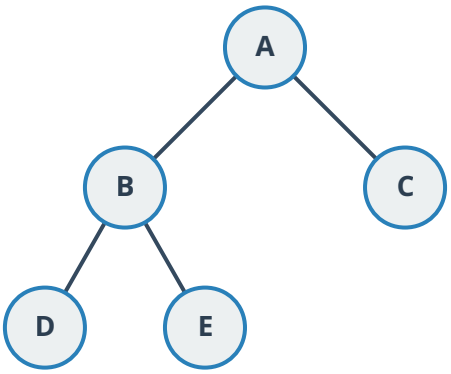
Notice

- `fib(3)` computed twice
- `fib(2)` computed three times

This is called overlapping subproblems.
We solve it using memoization.

Does this happen in a tree?

Consider



Suppose we call `dfs(A)`. The calls are:

- How many times is `dfs(D)` called? **Exactly once.**
- How many times is `dfs(E)` called? **Exactly once.**
- How many times is `dfs(B)` called? **Exactly once.**

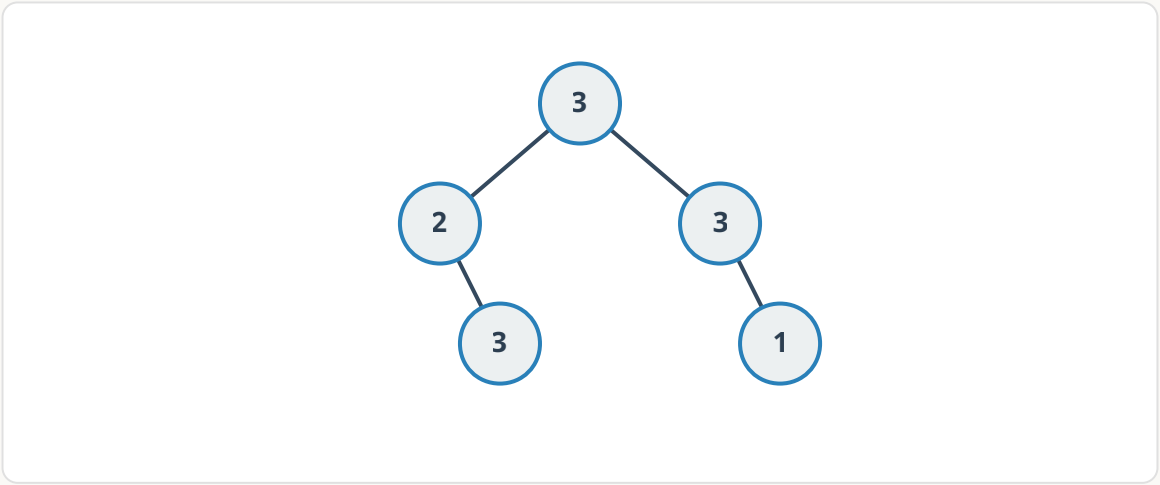
There is no second path to reach B.
Why?
Because a tree has:

- One parent
- One path from root

There are no cycles. There are no multiple parents.
Therefore, every subtree is solved exactly once.

House Robber III

Suppose



Calls:
`dfs(root) → dfs(2) → dfs(3) → return`

Every node is visited **ONE TIME**. No repeated computation.

Then why is this called DP?

This is one of the most common interview questions.
DP is not only about overlapping subproblems. DP has two properties:

1. Optimal Substructure &✅;
2. Overlapping Subproblems (sometimes)

House Robber III satisfies only the first one.

Property 1: Optimal Substructure
The answer for `root` depends on optimal answers of `left subtree` and `right subtree`.
This is exactly DP thinking.

Property 2: Overlapping Subproblems
Does the same subtree get solved again? No.
Then why don't we call it recursion?
Because every recursive call is returning a DP state.
Notice `pair<int,int> dfs(node)` doesn't simply traverse.
It computes `DP(node)` which is `{notRob, rob}`.
So this is **DP using DFS** or **Tree DP**.

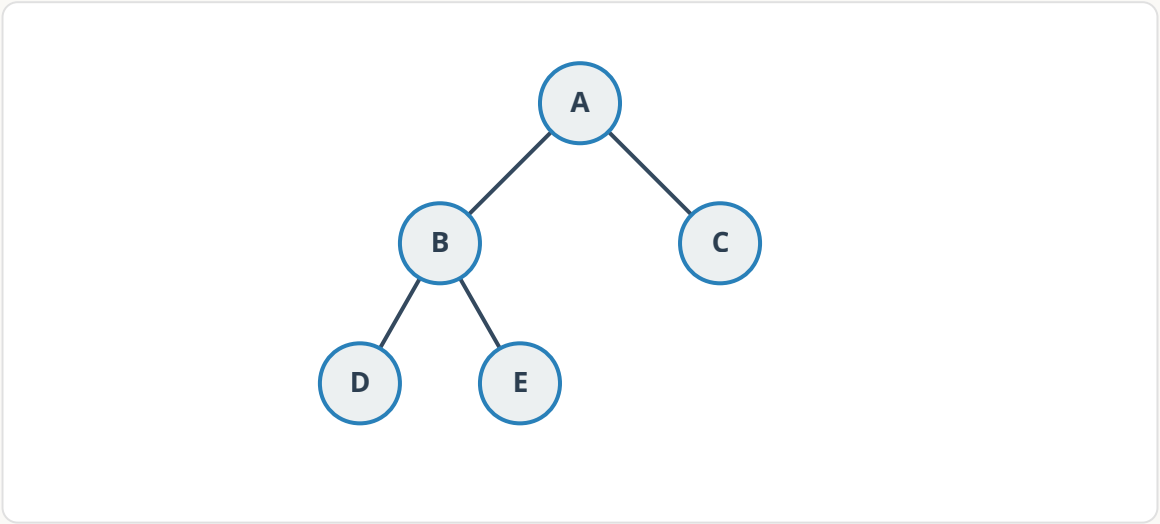
Can House Robber III have overlapping subproblems?

Yes, but only in a different solution.

Many beginners write

```
rob(node)
{
    rob current
    +
    rob(grandchildren)

    OR

    rob(children)
}
```

Example



Suppose `rob(A)` calls `rob(B)`, `rob(C)`, `rob(D)`, `rob(E)`.

Now `rob(B)` will again call `rob(D)`, `rob(E)`.

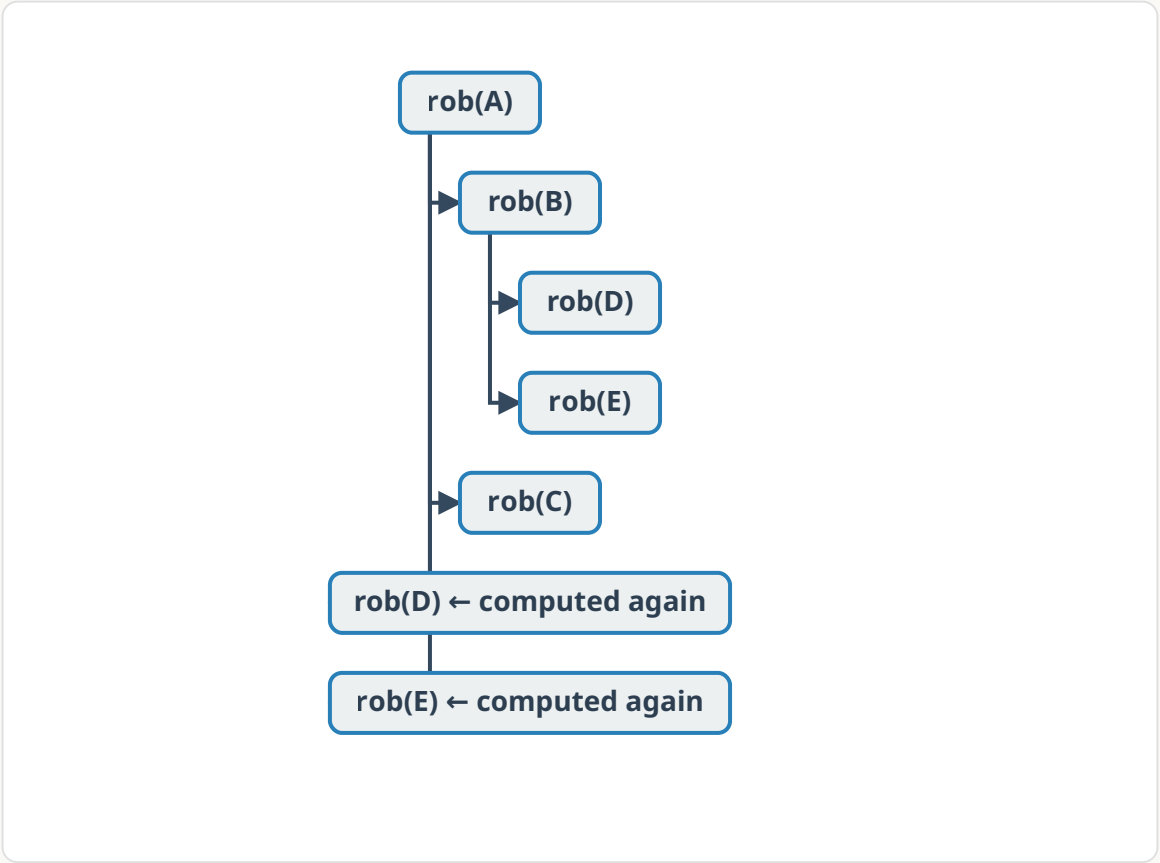
Now `D` and `E` are computed twice.

This creates **Overlapping Subproblems**.

This recursive solution is exponential, and you fix it by memoizing each node.

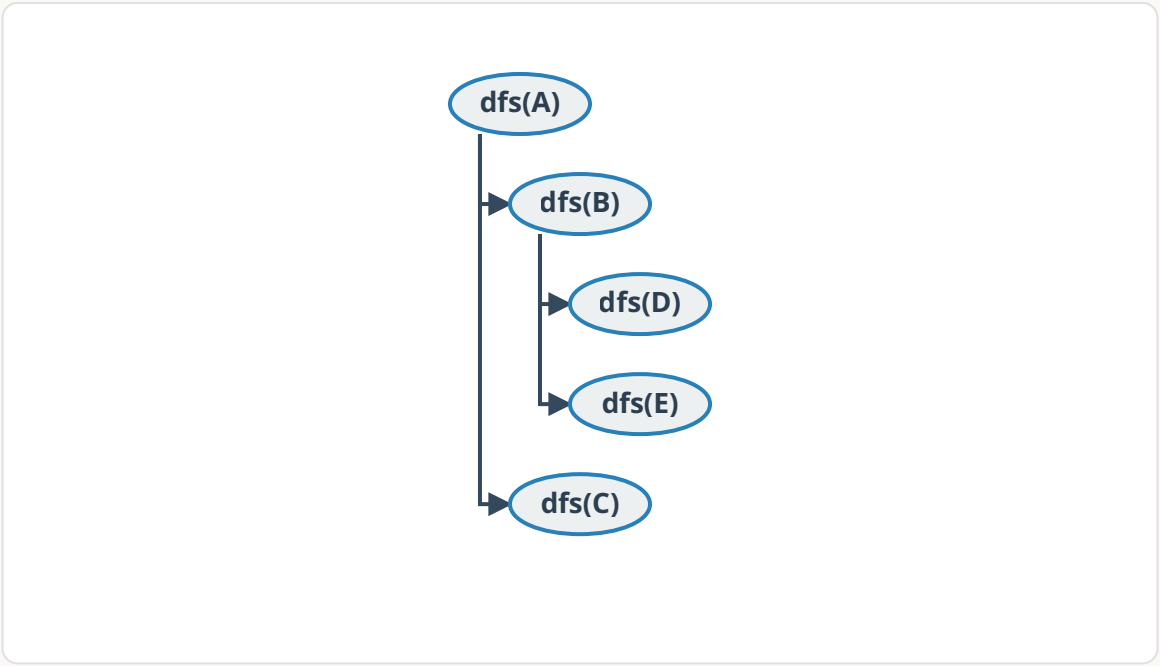
Comparison

1. Naive Recursive Solution



Here, `rob(D)` and `rob(E)` are repeated. Need memoization.
Time `O(2^N)` (or exponential due to repeated subtree evaluations)

2. Optimal Tree DP



Every node visited exactly once.
Time `O(N)`

Interview Takeaway

If an interviewer asks:

"Where are the overlapping subproblems in House Robber III?"

A strong answer is:

> In the optimal postorder Tree DP solution, there are no overlapping subproblems because every subtree is evaluated exactly once. The DP state is propagated upward in a single DFS traversal. However, the naive recursive solution that separately explores robbing children and grandchildren recomputes the same subtrees many times, creating overlapping subproblems. Memoization removes those repeated computations, and the pair-based Tree DP formulation eliminates them entirely by computing each node's state exactly once.

This distinction is important because it shows you understand why the pair-returning Tre